

Trees versus Neural Networks on Tabular Data

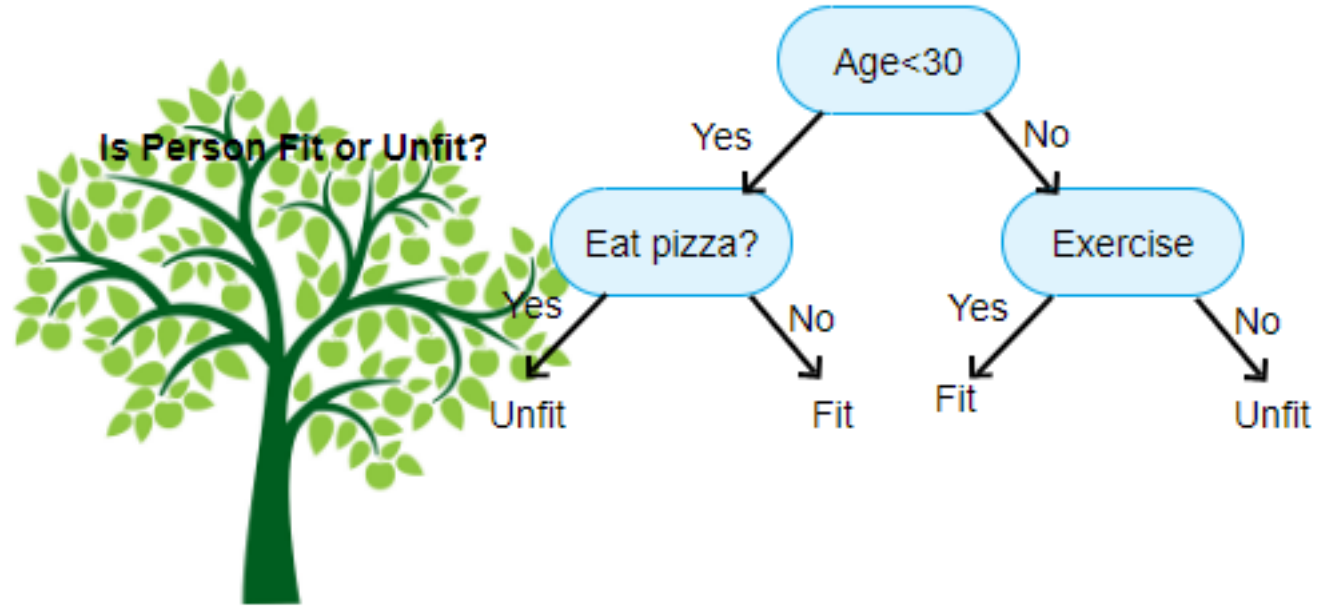
Troy Pinkney

Project Objective

To compare tree-based models (trees) with deep neural networks (DNNs) on tabular data

Trees

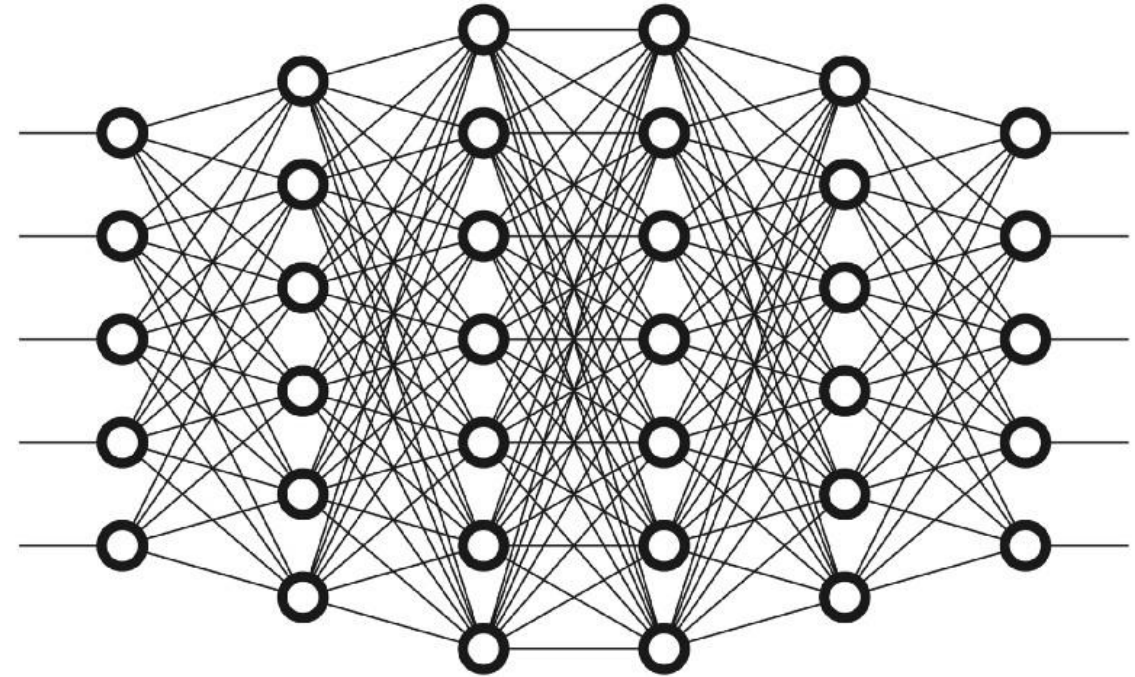
- Works by asking questions
- Tabular data de-facto model
- Famous for winning competitions



Source: <https://medium.com/@sachinsoni600517/mastering-decision-tree-models-for-smarter-decisions-451ef8b75836>

DNNs

- Sea of interconnecting layers and nodes
- Poor in tabular domain
- De-facto model every other domain



Source: <https://www.freecodecamp.org/news/want-to-know-how-deep-learning-works-heres-a-quick-guide-for-everyone-1aedeca88076/>

Tabular Dataset Example – California Housing

Feature	Values
House Value (Target)	\$
Income	\$
Total Rooms	0 to 39000
Distance to coast	meters
Distance to LA	meters
Distance to San Francisco	meters
Distance to San Diego	meters
Households	1 to 6082
...	...

Why is this project important?

- ML in tabular domain
 - Healthcare – disease diagnosis
 - Finance – credit card fraud
 - Retail – inventory management
- Interpretable models (trees)
- Want DNN architecture in tabular domain
- Low resource consumption model (TabPFN)
 - Accessible to wider audience
 - Environmentally friendly
 - Cheaper
 - Less invasion of privacy

Project Objective - Steps

- 1) Gather datasets from Kaggle and UCI
- 2) Default hyper-parameter benchmark
- 3) Hyper-parameter optimization techniques
- 4) DNN Transformer TabPFN

What I have done

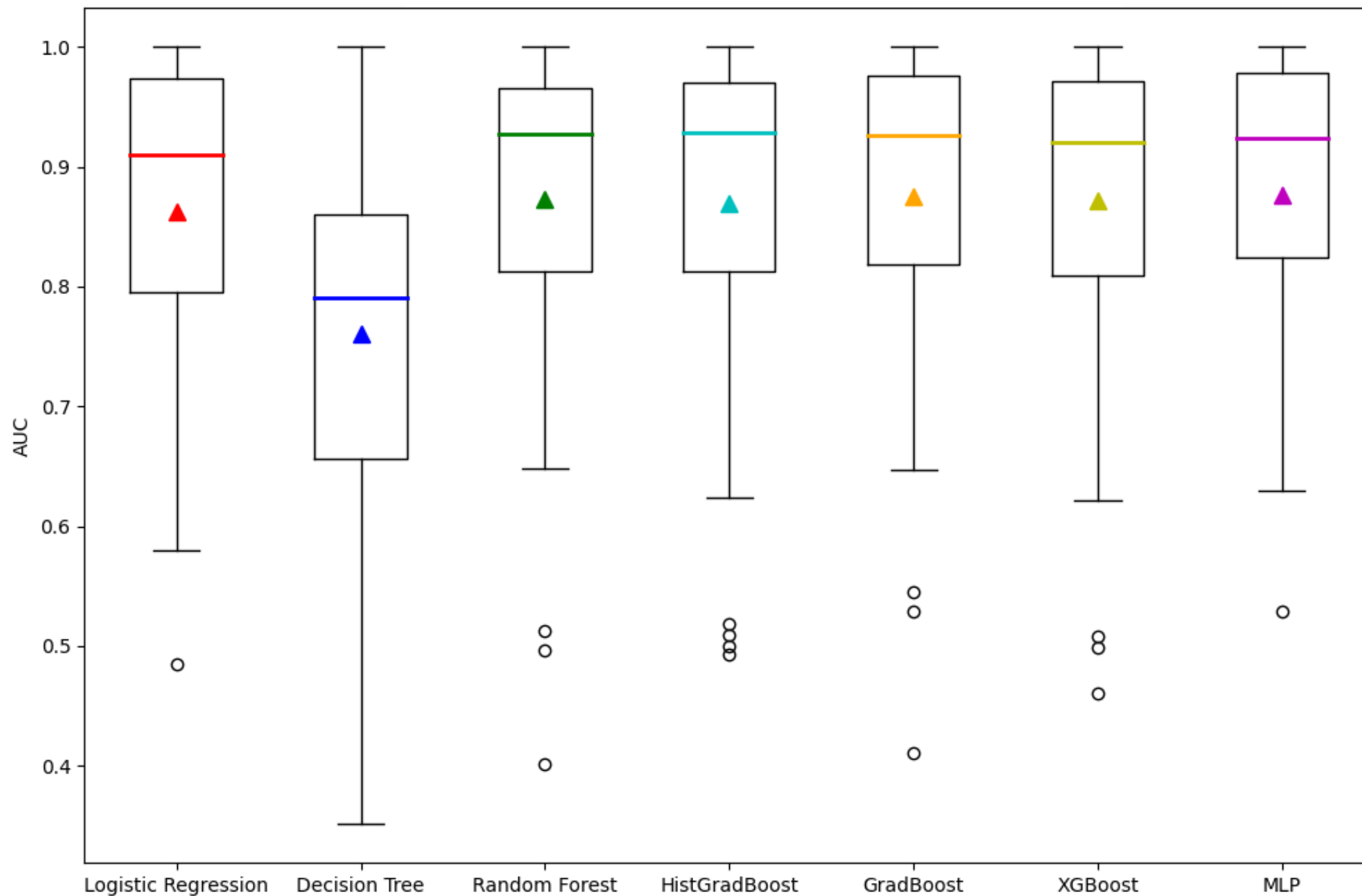
- Collected 44 datasets (original datasets)
 - Compared 1 baseline model, 5 trees and 1 DNN
 - Default hyper-parameter benchmark
 - Hyperparameter tuning benchmark via random search and Bayesian optimization
- Generated 29 small datasets (trimmed datasets) by trimming the original datasets
 - Default hyper-parameter benchmark
 - Hyper-parameter tuning benchmark with TabPFN

Original Datasets – General Info

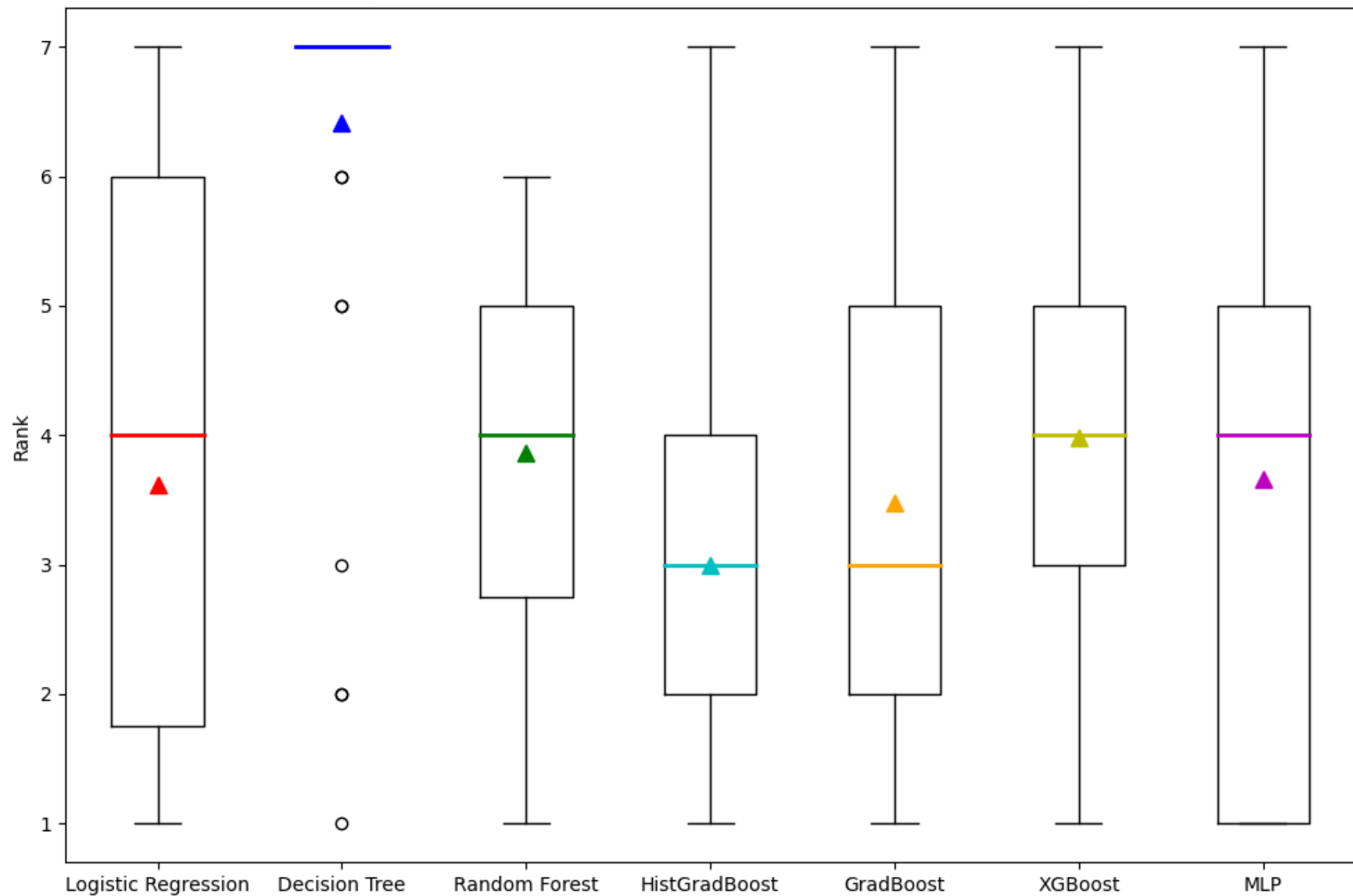
Category	Mean Value
Majority Class %	58.9%
Number of Features	26.5
Number of Samples	3472.2

Original Datasets: Default Benchmark

Original Datasets: Default Hyper-parameter AUC Plot

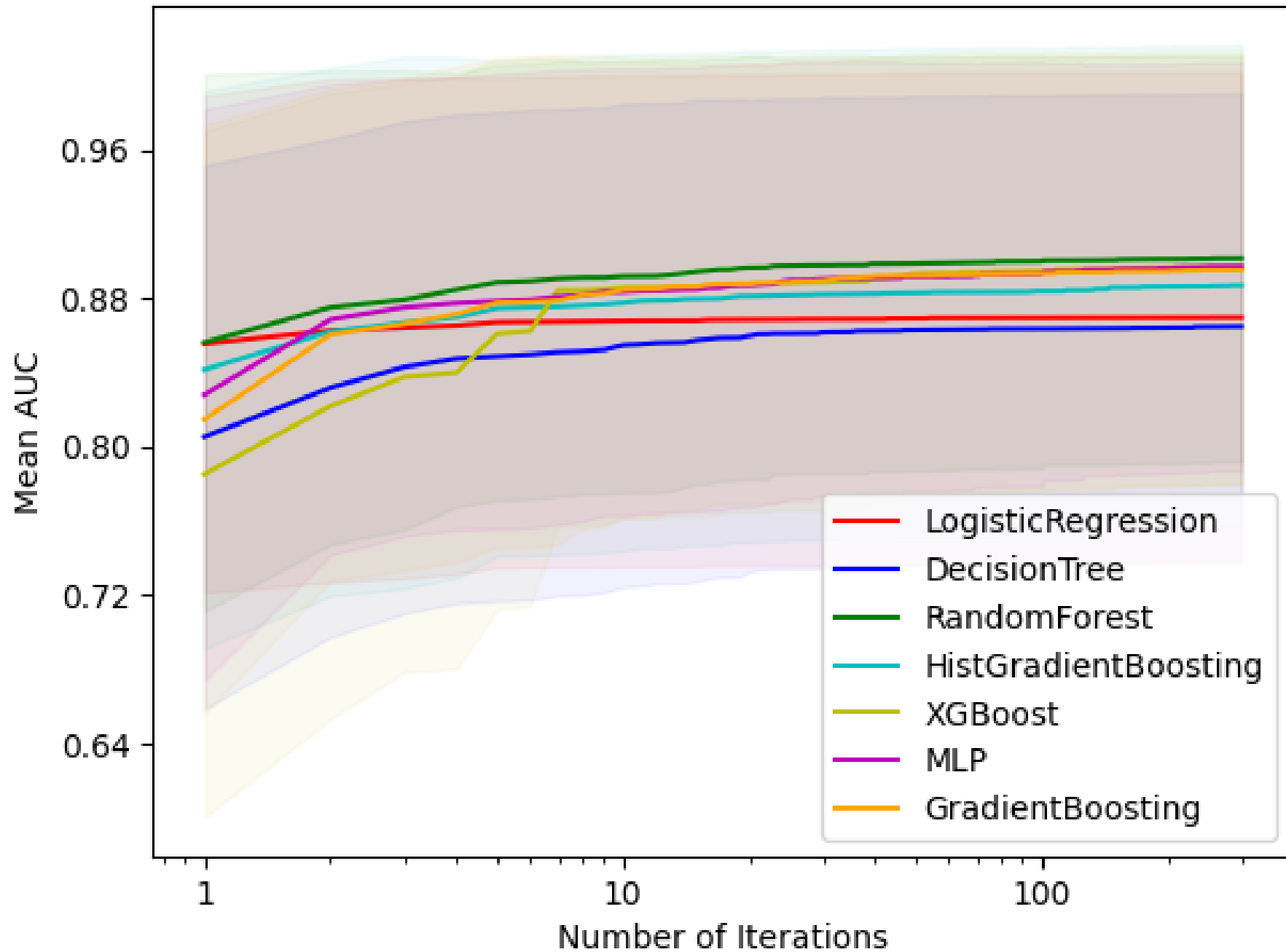


Original Datasets: Default Hyper-parameter Rank Plot

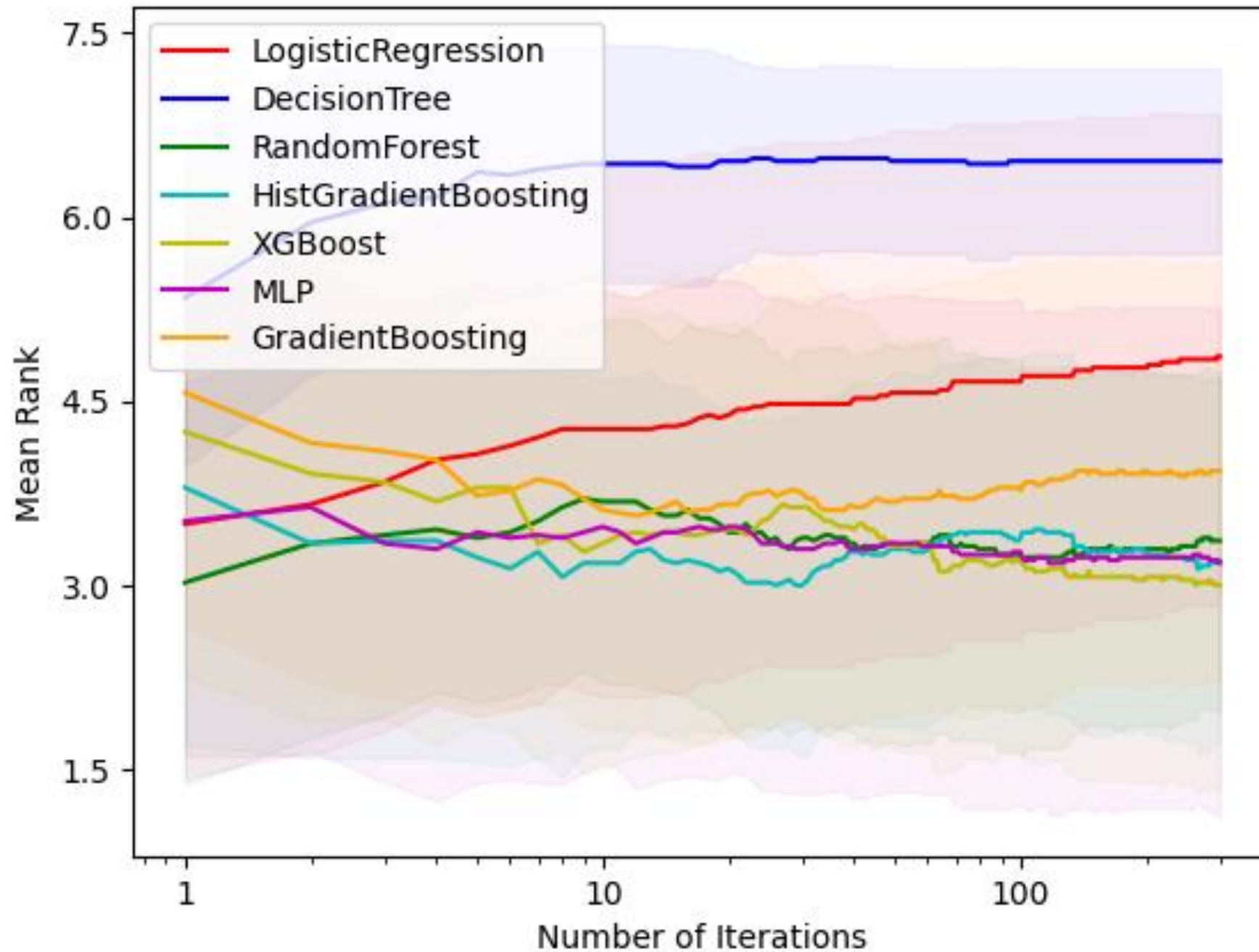


Original Datasets: Random Search Benchmark

Original Datasets: Random Search AUC Plot

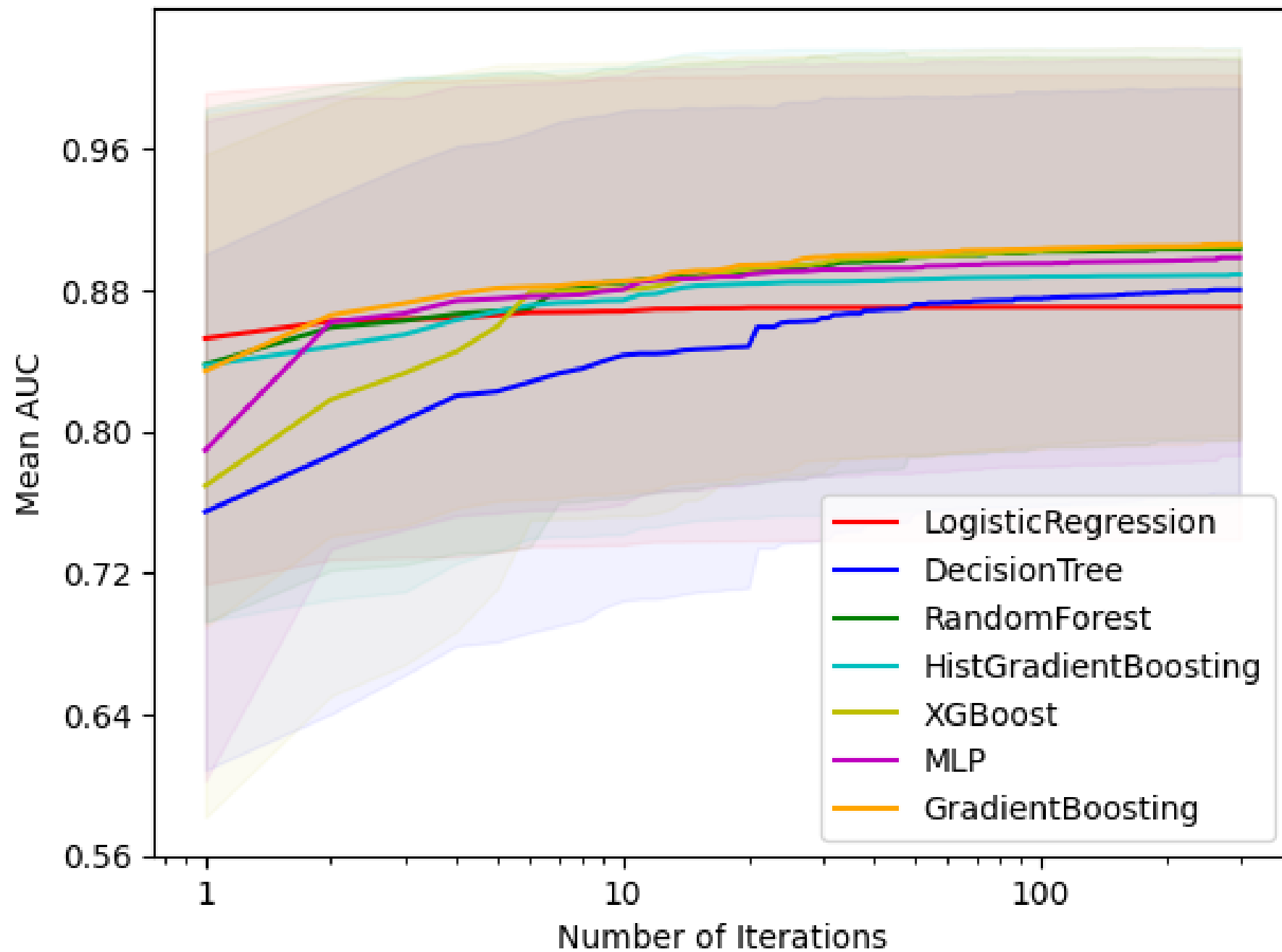


Original Datasets: Random Search Rank Benchmark

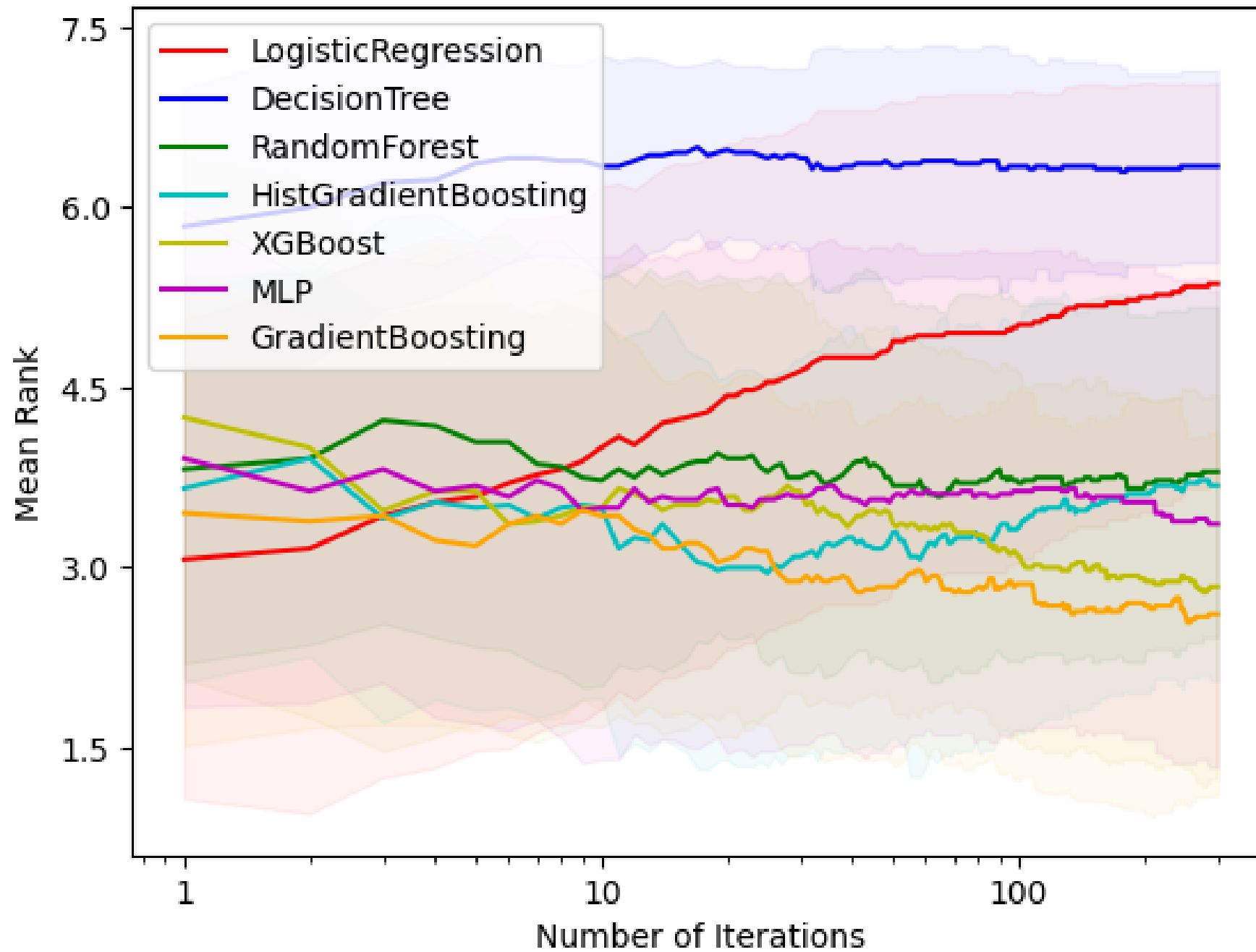


Original Datasets: Bayesian Optimization

Original Datasets: Bayesian Optimization AUC Plot



Original Datasets: Bayesian Optimization Rank Plot

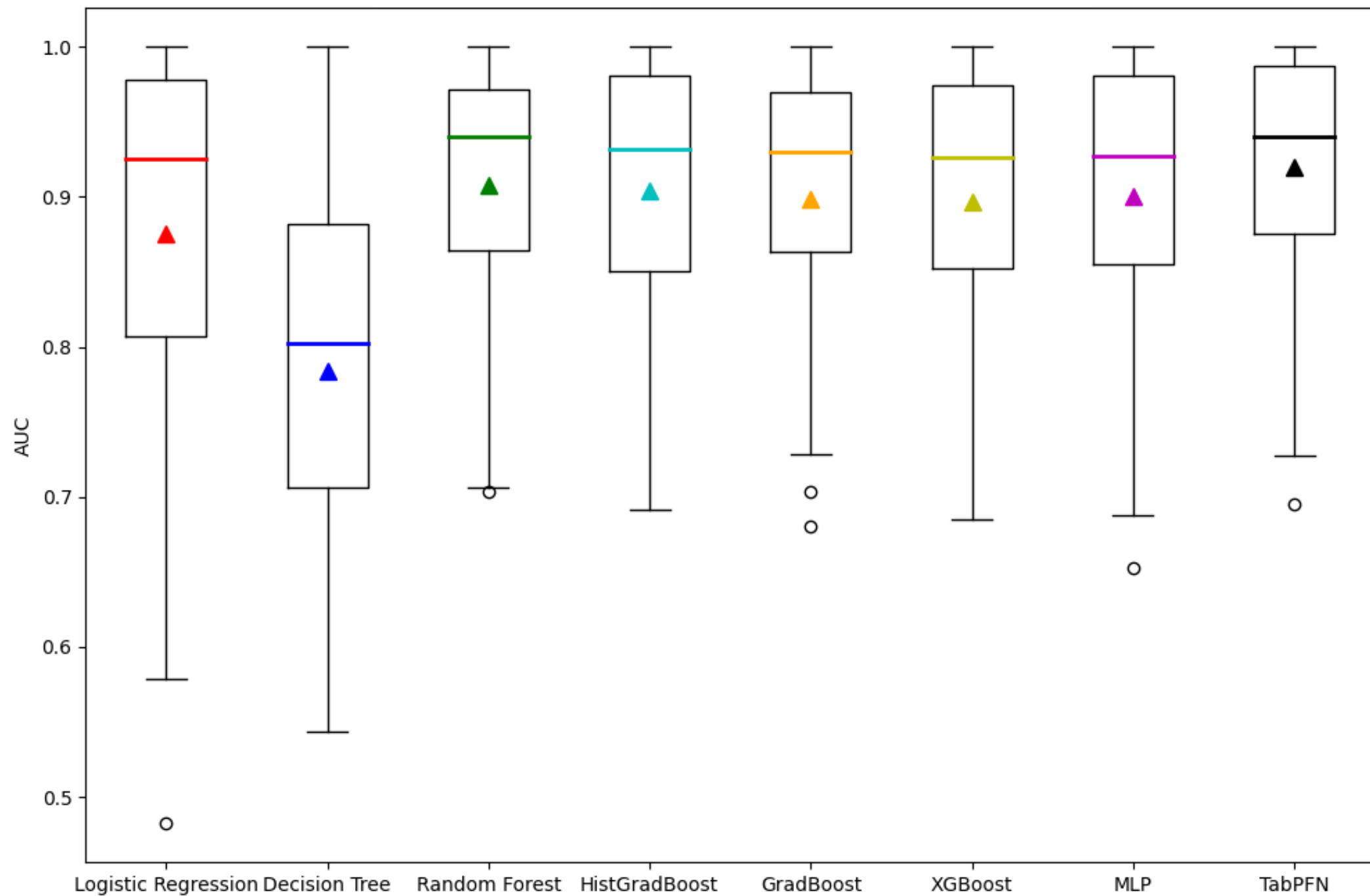


Trimmed Datasets – General Info

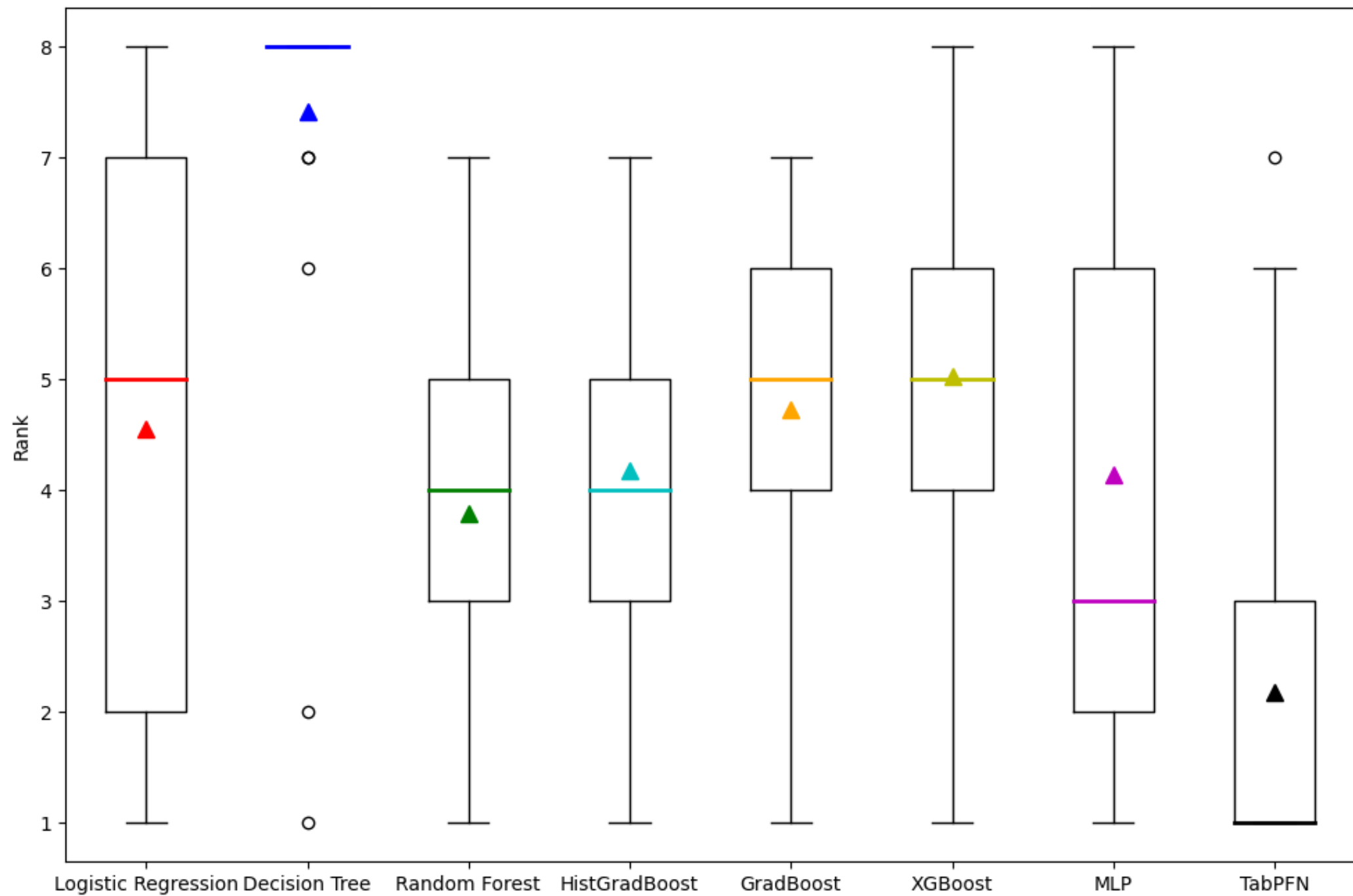
Category	Mean Value
Majority Class %	55.3%
Number of Features	17.58
Number of Samples	1023.3

Trimmed Datasets: Default Benchmark

Trimmed Datasets: Default Hyper-parameter AUC Plot

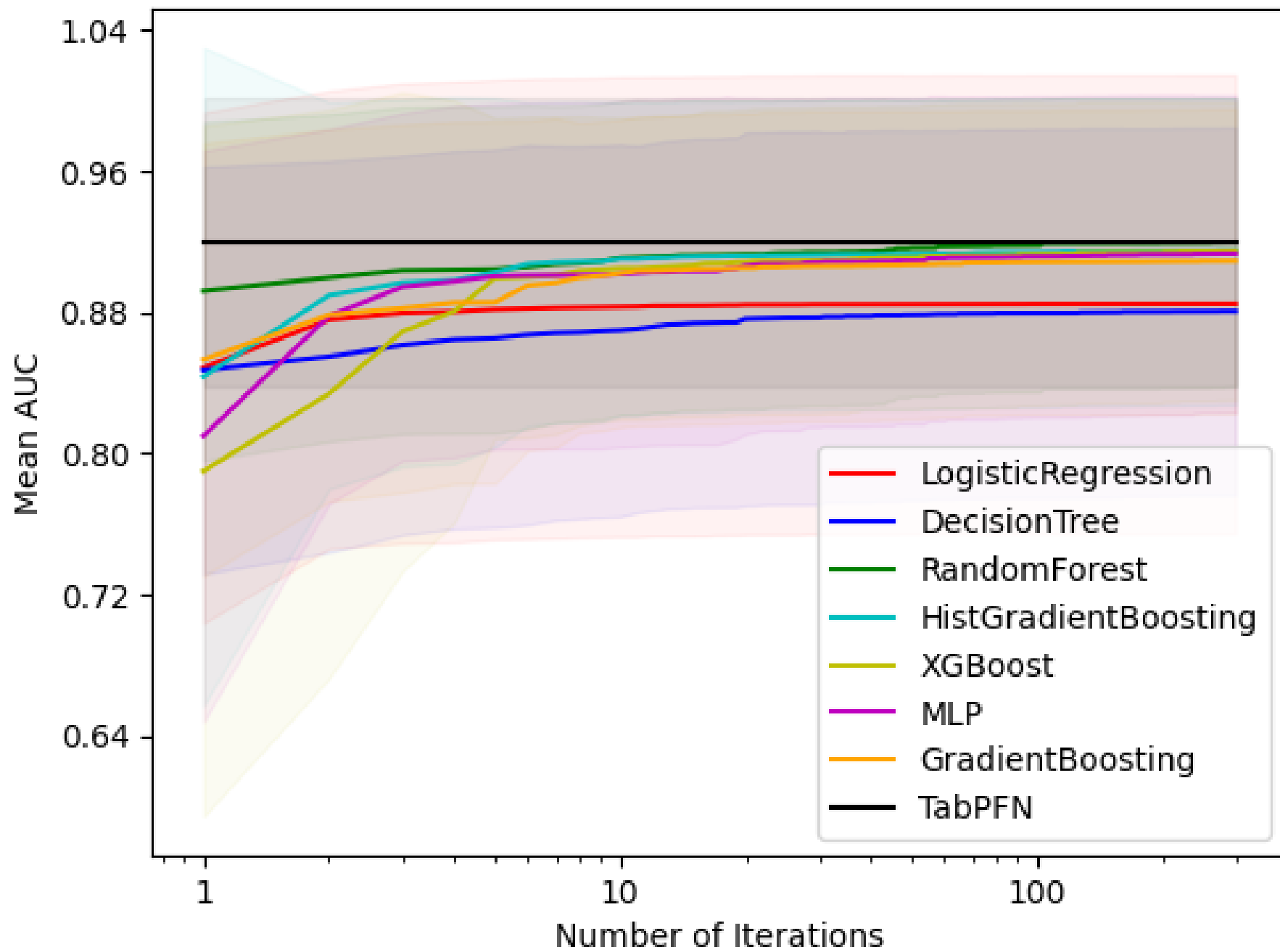


Trimmed Datasets: Default Hyper-parameter Rank Plot

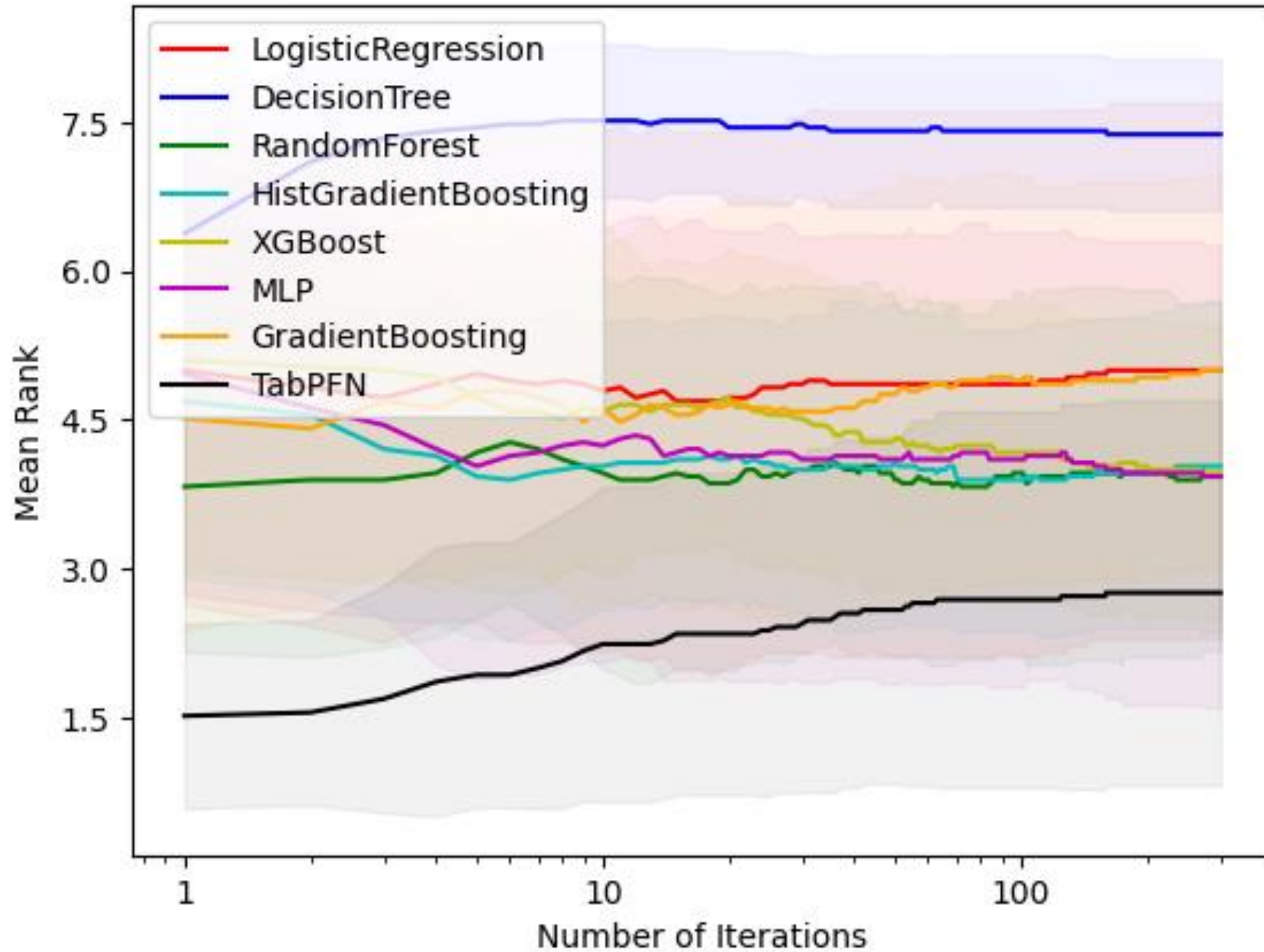


Trimmed Datasets: Random Search

Trimmed Datasets: Random Search AUC Plot



Trimmed Datasets: Random Search Rank Plot



Summary

- Original datasets – default, random search, Bayesian optimization : trees marginally better
- Trimmed datasets – default, random search: TabPFN (DNNs) significantly better

Conclusion

- Trees vs DNNs is size of dataset and individual dataset dependent
- Superiority margin invariant to tuning (except TabPFN)

Q&A

Thanks for Listening!